Day 3 — Single-Agent Systems (ReAct)

> 🎯 **Goal**: Build a reliable, tool-using Agent.

> ⏱️ **Time**: 1–1.5h theory + 3h practice

> 📦 **Deliverable**: A functioning single-agent ReAct system with tool execution

---

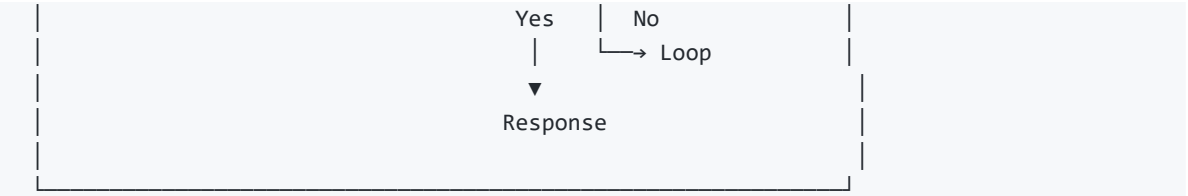
Table of Contents

1. What Is an Agent?
2. The ReAct Pattern
3. Agent Memory Types
4. Why Agents Fail
5. Practice: Build a ReAct Agent

What Is an Agent?

An Agent is an LLM that can **decide what actions to take**, **execute those actions**, and **use the results to inform its next steps** — autonomously.





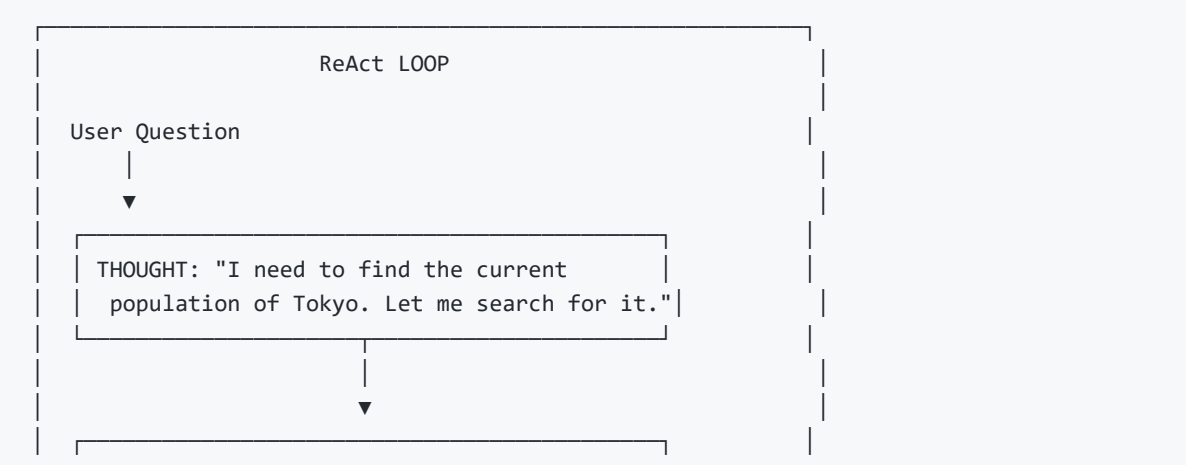
Key Differences

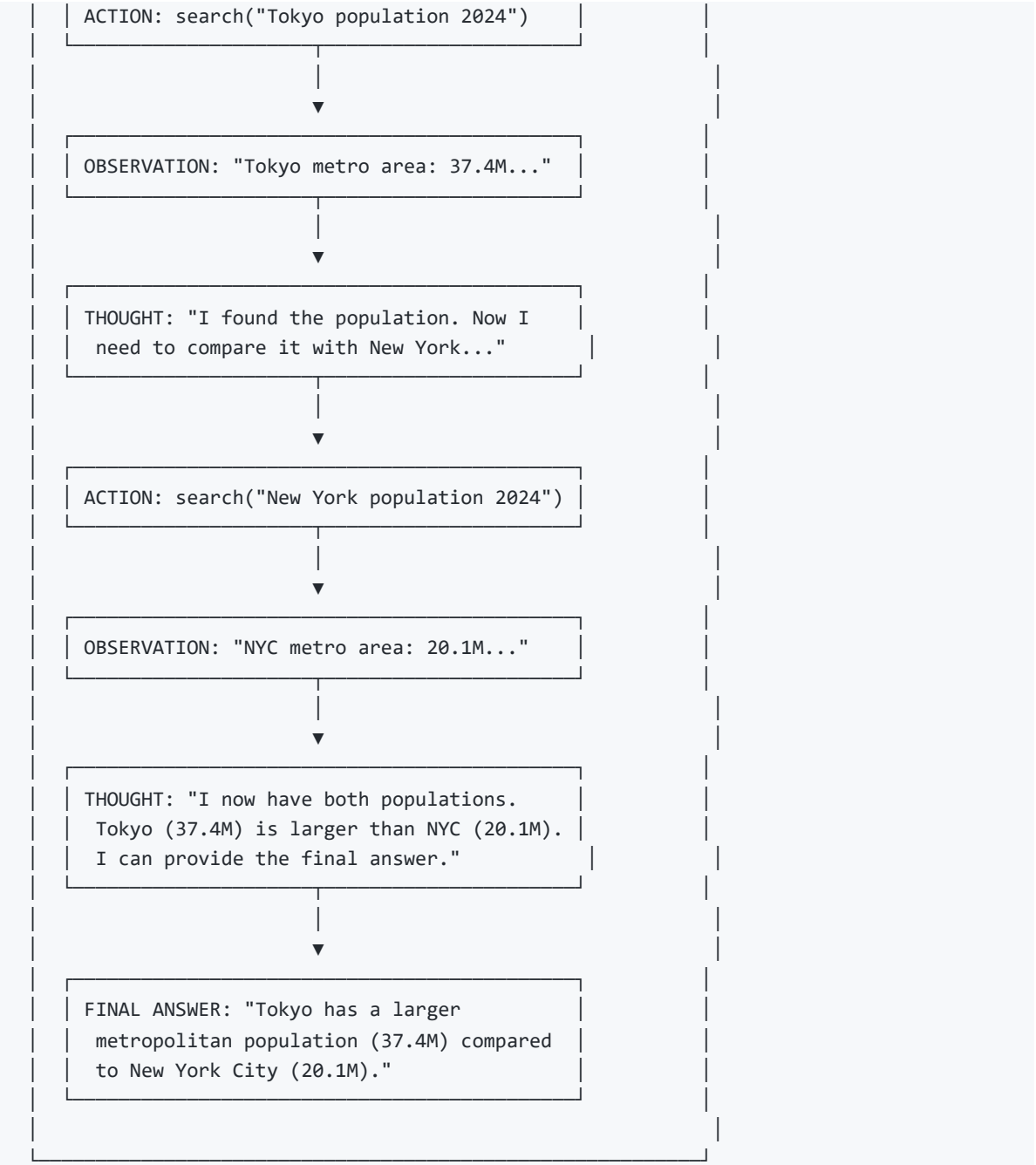
Feature	Chatbot	Agent
Tools	None	Calculator, APIs, databases, code execution
Planning	None	Multi-step task decomposition
Memory	Conversation history only	Short-term + long-term + episodic
Autonomy	Responds to each message	Decides what to do next
Loops	Single turn	Multiple think→act→observe cycles
Error handling	None	Self-correction, retries

The ReAct Pattern

ReAct (Reason + Act) is the most important agent pattern. Published by Yao et al. (2022), it interleaves **reasoning** (thinking about what to do) with **acting** (using tools) and **observing** (processing results).

The ReAct Loop





Why ReAct Works

```
Without ReAct (pure reasoning):
  Q: "What's 47 × 83 + 156?"
  A: "4,057" ← The LLM does mental math (often wrong for large numbers)

With ReAct (reason + act):
  Thought: "I need to calculate 47 × 83 first, then add 156.
            Let me use the calculator tool."
  Action: calculator("47 * 83")
  Observation: 3901
  Thought: "Now I need to add 156."
  Action: calculator("3901 + 156")
  Observation: 4057
```

```
  Answer: "47 × 83 + 156 = 4,057"

The key insight: Let the LLM REASON about what to do,
but let TOOLS do the actual work.
```

ReAct vs Other Patterns

```
┌─────────────────────────────────────────────────────────┐
│                    AGENT PATTERNS                       │
│                                                         │
│  1. REASON-ONLY (Chain-of-Thought)                      │
│     Think → Think → Think → Answer                      │
│     Problem: Can't access real-time data or compute     │
│                                                         │
│  2. ACT-ONLY (Tool chaining)                            │
│     Tool → Tool → Tool → Answer                         │
│     Problem: No reasoning about which tool to use       │
│                                                         │
│  3. ReAct (Reason + Act)  ← BEST                        │
│     Think → Act → Observe → Think → Act → Observe → Ans │
│     Reasoning guides tool use, observations guide next  │
│     reasoning step                                      │
│                                                         │
│  4. PLAN-then-EXECUTE (Day 4)                           │
│     Plan all steps → Execute sequentially → Review      │
│     Better for complex multi-step tasks                 │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

Agent Memory Types

Agents need memory to operate effectively. There are three types:

1. Short-Term Memory (Scratchpad)

```
What: The current conversation / reasoning trace
Where: In the LLM's context window
Lifespan: Current task only
Size: Limited by context window (4K–128K tokens)

Example:
  The agent's scratchpad during a ReAct loop:
  ┌──────────────────────────────────────────┐
  │ Thought 1: I need to find the weather    │
  │ Action 1: get_weather("Tokyo")           │
  │ Observation 1: 22°C, sunny               │
  │ Thought 2: Now I need the forecast...    │  ← This IS short-term memory
  │ Action 2: get_forecast("Tokyo", days=3)  │
```

```
  │ Observation 2: [22, 25, 20]                 │
  │ Thought 3: I have all the information...    │
  └─────────────────────────────────────────────┘

Trade-off: Easy to implement but context window is finite.
  Llama 3 8B: 8K context → ~20 ReAct steps max
  GPT-4: 128K context → ~300 ReAct steps max
  But cost increases linearly with context length!
```

2. Long-Term Memory (Vector Store)

```
What: Persistent knowledge the agent can search
Where: External vector database (ChromaDB, Pinecone, FAISS)
Lifespan: Persists across sessions
Size: Unlimited (GBs of documents)

How it works:
  1. Documents → split into chunks → embed → store in vector DB
  2. Agent queries: "What did we discuss about authentication?"
  3. Vector DB returns most similar chunks
  4. Agent uses retrieved chunks as context

  ┌──────────────────────────────────────────────────────┐
  │                  LONG-TERM MEMORY                    │
  │                                                      │
  │  Agent query: "authentication best practices"        │
  │       │                                               │
  │       ▼                                               │
  │  Embed query → [0.23, 0.87, -0.12, ...]              │
  │       │                                               │
  │       ▼                                               │
  │  ┌──────────────────────────────────────┐            │
  │  │          VECTOR DATABASE             │            │
  │  │                                      │            │
  │  │  Doc 1: "Use JWT tokens for..."  0.92  │ ← Similar │
  │  │  Doc 2: "OAuth 2.0 flow: ..."   0.87  │ ← Similar │
  │  │  Doc 3: "Database indexes..."   0.23  │ ← Not     │
  │  │  Doc 4: "Rate limiting with..." 0.31  │ ← Not     │
  │  └──────────────────────────────────────┘            │
  │                                                      │
  │  Return top-K similar documents → inject into context │
  └──────────────────────────────────────────────────────┘

We'll build this fully on Day 5 (RAG).
```

3. Episodic Memory

```
What: Memory of past actions, decisions, and outcomes
Where: Structured log / database
Lifespan: Persists across sessions
Size: Grows over time
```

```
Purpose: Learn from past mistakes and successes

Example:
  ┌──────────────────────────────────────────────────────┐
  │               EPISODIC MEMORY LOG                    │
  │                                                      │
  │  Episode 1 (2024-01-15):                             │
  │    Task: "Deploy to production"                      │
  │    Actions: [run_tests, build, deploy]               │
  │    Outcome: FAILED — tests passed but deploy timed out│
  │    Lesson: "Increase deploy timeout to 300s"         │
  │                                                      │
  │  Episode 2 (2024-01-16):                             │
  │    Task: "Deploy to production"                      │
  │    Actions: [run_tests, build, deploy(timeout=300)]  │
  │    Outcome: SUCCESS                                  │
  │    Lesson: "300s timeout works for production deploy" │
  │                                                      │
  │  When agent gets "Deploy to production" again:       │
  │  → Retrieves episodes → Uses 300s timeout from start │
  └──────────────────────────────────────────────────────┘
```

Memory Comparison

Type	Location	Lifespan	Retrieval	Use Case
Short-term	Context window	Current task	Automatic (in prompt)	Reasoning steps, current state
Long-term	Vector DB	Permanent	Similarity search	Knowledge base, documents
Episodic	Database/log	Permanent	Task-based lookup	Learning from past actions

---

Why Agents Fail

Understanding failure modes is critical for building reliable agents.

Failure 1: Infinite Loops

```
What happens:
  Thought: "I need to search for X"
  Action: search("X")
  Observation: "No results found"
  Thought: "Let me try searching for X"     ← SAME action again!
  Action: search("X")
```

```
  Observation: "No results found"
  Thought: "Let me search for X"            ← Stuck in a loop!
  ... forever ...

Why it happens:
  - The model doesn't recognize it already tried this
  - No mechanism to try alternative approaches
  - Same failed action feels "right" to the model

Prevention:
  1. Track action history → reject duplicate actions
  2. Set a maximum number of steps (e.g., max_steps=10)
  3. After N failures, force the agent to try a different approach
  4. Include action history in the prompt
```

Failure 2: Hallucinated Tools

```
What happens:
  Thought: "Let me use the email_sender tool"
  Action: email_sender("user@example.com", "Hello!")

  Problem: email_sender doesn't exist! The model invented it.

Why it happens:
  - The model has seen many tools in training data
  - It "remembers" tools that aren't available
  - Similar to how models hallucinate facts

Prevention:
  1. Explicitly list available tools in the system prompt
  2. Validate tool names before execution
  3. Return clear error: "Tool 'email_sender' not available.
     Available tools: calculator, search, get_weather"
  4. Let the agent recover from the error
```

Failure 3: Malformed Tool Arguments

```
What happens:
  Action: calculator("forty-seven times eighty-three")

  Problem: Calculator expects "47 * 83", not English words

Why it happens:
  - Model doesn't perfectly follow argument schemas
  - Especially with complex or nested arguments

Prevention:
  1. Provide clear argument specifications with examples
  2. Validate arguments before execution
```

```
  3. Use JSON schemas for tool arguments
  4. On error, show the agent the schema and ask it to retry
```

Failure 4: Premature Termination

```
What happens:
  User: "Find the 3 cheapest flights from NYC to Tokyo next month"
  Thought: "Let me search for flights"
  Action: search_flights("NYC", "Tokyo", "2024-02-01")
  Observation: [Flight A: $800, Flight B: $950, Flight C: $1100]
  Answer: "The cheapest flight is $800 on Flight A."

  Problem: Only reported 1 flight, user asked for 3!

Why it happens:
  - Model "thinks" it's done when it has a partial answer
  - Doesn't check its answer against the original question

Prevention:
  1. Include the original question in every reasoning step
  2. Add a validation step: "Does my answer fully address the question?"
  3. Use a separate critic prompt to check completeness
```

Anti-Pattern Summary

```
┌──────────────────┬──────────────────────────────────────────┐
│ Anti-Pattern     │ Solution                                 │
├──────────────────┼──────────────────────────────────────────┤
│ Infinite loops   │ Max steps + duplicate detection + fallback │
│ Tool hallucin.   │ Strict tool registry + validation        │
│ Bad arguments    │ Schema validation + retry with error msg │
│ Early stopping   │ Answer validation + critic check         │
│ Context overflow │ Summarize old steps + truncation strategy │
│ Wrong tool pick  │ Better tool descriptions + examples      │
└──────────────────┴──────────────────────────────────────────┘
```

Practice: Build a ReAct Agent

Complete ReAct Agent Implementation

```
  """
  ReAct Agent — A complete implementation with tool use,
  validation, and loop prevention.

  This agent:
```

```
  1. Creates a multi-step plan
  2. Selects and uses tools
  3. Validates its reasoning
  4. Avoids action loops
  5. Handles errors gracefully
"""
import json
import re
import math
import urllib.request
import urllib.parse
from dataclasses import dataclass, field
from typing import Callable


# ═══════════════════════════════════════════════════════════════
# SECTION 1: TOOL DEFINITIONS
# ═══════════════════════════════════════════════════════════════

@dataclass
class Tool:
    """A tool the agent can use."""
    name: str
    description: str
    parameters: dict  # JSON schema for parameters
    function: Callable

    def to_prompt_description(self) -> str:
        """Format tool info for the agent's system prompt."""
        params_str = json.dumps(self.parameters, indent=2)
        return (
            f"Tool: {self.name}\n"
            f"Description: {self.description}\n"
            f"Parameters: {params_str}\n"
        )


def calculator(expression: str) -> str:
    """Safely evaluate a mathematical expression."""
    # Allow only safe math operations
    allowed_chars = set("0123456789+-*/().% ")
    allowed_funcs = {"sqrt": math.sqrt, "pow": pow, "abs": abs,
                     "round": round, "min": min, "max": max}

    if not all(c in allowed_chars or c.isalpha() for c in expression):
        return f"Error: Invalid characters in expression: {expression}"

    try:
        # Replace common math functions
        expr = expression
        for name, func in allowed_funcs.items():
            expr = expr.replace(name, f"__{name}__")

        # Evaluate safely
        result = eval(expr, {"__builtins__": {}},
                      {f"__{k}__": v for k, v in allowed_funcs.items()})
```

```
        return str(result)
    except Exception as e:
        return f"Error: Could not evaluate '{expression}': {e}"


def get_weather(city: str) -> str:
    """Get weather for a city (simulated for practice)."""
    # In production, call a real API like OpenWeatherMap
    weather_data = {
        "tokyo": {"temp": 22, "condition": "Sunny", "humidity": 45},
        "new york": {"temp": 15, "condition": "Cloudy", "humidity": 65},
        "london": {"temp": 12, "condition": "Rainy", "humidity": 80},
        "paris": {"temp": 18, "condition": "Partly Cloudy", "humidity": 55},
        "sydney": {"temp": 28, "condition": "Sunny", "humidity": 40},
    }

    city_lower = city.lower().strip()
    if city_lower in weather_data:
        data = weather_data[city_lower]
        return json.dumps({
            "city": city,
            "temperature_celsius": data["temp"],
            "condition": data["condition"],
            "humidity_percent": data["humidity"]
        })

    return json.dumps({"error": f"Weather data not available for '{city}'. "
                       f"Available cities: {', '.join(weather_data.keys())}"})


def search(query: str) -> str:
    """Search for information (simulated knowledge base)."""
    # In production, use a real search API (Google, Bing, etc.)
    knowledge = {
        "population tokyo": "Tokyo metropolitan area has a population of approximate
        "population new york": "New York City metropolitan area has a population of
        "python creator": "Python was created by Guido van Rossum and first released
        "speed of light": "The speed of light in vacuum is approximately 299,792,458
        "largest ocean": "The Pacific Ocean is the largest ocean, covering approxima
        "bitcoin creator": "Bitcoin was created by an anonymous person or group usin
    }

    query_lower = query.lower().strip()

    # Find best match
    best_match = None
    best_score = 0
    for key, value in knowledge.items():
        # Simple keyword matching
        query_words = set(query_lower.split())
        key_words = set(key.split())
        overlap = len(query_words & key_words)
        if overlap > best_score:
            best_score = overlap
            best_match = value
```

```
    if best_match and best_score > 0:
        return best_match

    return f"No results found for '{query}'. Try different search terms."


def string_utility(operation: str, text: str) -> str:
    """Perform string operations."""
    operations = {
        "length": lambda t: str(len(t)),
        "upper": lambda t: t.upper(),
        "lower": lambda t: t.lower(),
        "reverse": lambda t: t[::-1],
        "word_count": lambda t: str(len(t.split())),
    }

    if operation not in operations:
        return f"Unknown operation '{operation}'. Available: {', '.join(operations.k

    return operations[operation](text)


# Define tools with their schemas
TOOLS = {
    "calculator": Tool(
        name="calculator",
        description="Evaluate mathematical expressions. Supports +, -, *, /, %, sqrt
        parameters={
            "type": "object",
            "required": ["expression"],
            "properties": {
                "expression": {
                    "type": "string",
                    "description": "Math expression to evaluate, e.g., '47 * 83 + 15
                }
            }
        },
        function=lambda args: calculator(args["expression"])
    ),
    "search": Tool(
        name="search",
        description="Search for factual information. Use for questions about people,
        parameters={
            "type": "object",
            "required": ["query"],
            "properties": {
                "query": {
                    "type": "string",
                    "description": "Search query, e.g., 'population of Tokyo'"
                }
            }
        },
        function=lambda args: search(args["query"])
    ),
    "get_weather": Tool(
        name="get_weather",
```

```
            description="Get current weather for a city. Returns temperature, condition,
            parameters={
                "type": "object",
                "required": ["city"],
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "City name, e.g., 'Tokyo', 'New York'"
                    }
                }
            },
            function=lambda args: get_weather(args["city"])
        ),
        "string_utility": Tool(
            name="string_utility",
            description="Perform string operations: length, upper, lower, reverse, word_
            parameters={
                "type": "object",
                "required": ["operation", "text"],
                "properties": {
                    "operation": {
                        "type": "string",
                        "enum": ["length", "upper", "lower", "reverse", "word_count"],
                        "description": "The string operation to perform"
                    },
                    "text": {
                        "type": "string",
                        "description": "The text to operate on"
                    }
                }
            },
            function=lambda args: string_utility(args["operation"], args["text"])
        ),
    }


    # ═══════════════════════════════════════════════════════════════
    # SECTION 2: THE ReAct AGENT
    # ═══════════════════════════════════════════════════════════════

    class ReActAgent:
        """
        A ReAct agent that reasons and acts in an interleaved loop.

        Architecture:
        ┌─────────────────────────────────────────────┐
        │              ReAct Agent                    │
        │                                             │
        │  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
        │  │ LLM      │  │ Tool     │  │ Memory   │  │
        │  │ (reason) │  │ Registry │  │ (scratch-│  │
        │  │          │  │          │  │  pad)    │  │
        │  └────┬─────┘  └────┬─────┘  └────┬─────┘  │
        │       │             │             │        │
        │       └─────────────┴─────────────┘        │
        │                     │                       │
```

```
        │              Agent Loop                          │
        │         Think → Act → Observe                    │
        │                     │                            │
        │              Max Steps Guard                     │
        │              Loop Detection                      │
        └──────────────────────────────────────────────────┘
        """

        def __init__(self, tools: dict, model: str = "gpt-4", max_steps: int = 10):
            self.tools = tools
            self.model = model
            self.max_steps = max_steps
            self.scratchpad = []  # Short-term memory
            self.action_history = []  # For loop detection

        def _build_system_prompt(self) -> str:
            """Build the system prompt with tool descriptions."""
            tools_desc = "\n".join(
                tool.to_prompt_description() for tool in self.tools.values()
            )

            return f"""You are a helpful AI agent that solves tasks by reasoning step-by

    ## Available Tools
    {tools_desc}

    ## How to Respond

    You MUST respond in one of two formats:

    ### Format 1: Use a tool
```

Thought: Action: Action Input:

```
### Format 2: Give the final answer (when you have enough information)
```

Thought: <your reasoning about why you're done> Final Answer: <your complete answer to the user's question>

```
## Rules
1. Always start with a Thought explaining your reasoning
2. Use tools when you need external information or computation
3. NEVER make up information — use tools to get facts
4. If a tool returns an error, try a different approach
5. Do NOT repeat the same action with the same inputs
6. When you have enough information, provide the Final Answer
7. The Final Answer should fully address the user's original question"""

    def _parse_response(self, text: str) -> dict:
```

```
        """Parse the agent's response into structured components."""
        result = {"thought": "", "action": None, "action_input": None,
                  "final_answer": None}

        # Extract Thought
        thought_match = re.search(r'Thought:\s*(.*?)(?=\n(?:Action:|Final Answer:)|$)',
                                  text, re.DOTALL)
        if thought_match:
            result["thought"] = thought_match.group(1).strip()

        # Check for Final Answer
        final_match = re.search(r'Final Answer:\s*(.*)', text, re.DOTALL)
        if final_match:
            result["final_answer"] = final_match.group(1).strip()
            return result

        # Extract Action and Action Input
        action_match = re.search(r'Action:\s*(\w+)', text)
        if action_match:
            result["action"] = action_match.group(1).strip()

        input_match = re.search(r'Action Input:\s*(.*?)(?=\n\n|$)', text, re.DOTALL)
        if input_match:
            raw_input = input_match.group(1).strip()
            try:
                result["action_input"] = json.loads(raw_input)
            except json.JSONDecodeError:
                # Try to interpret as simple string argument
                result["action_input"] = {"query": raw_input}

        return result

    def _execute_tool(self, action: str, action_input: dict) -> str:
        """Execute a tool and return the result."""
        if action not in self.tools:
            available = ", ".join(self.tools.keys())
            return (f"Error: Tool '{action}' not found. "
                    f"Available tools: {available}")

        try:
            tool = self.tools[action]
            result = tool.function(action_input)
            return result
        except Exception as e:
            return f"Error executing {action}: {str(e)}"

    def _check_loop(self, action: str, action_input: dict) -> bool:
        """Check if this action has been tried before with same inputs."""
        action_key = f"{action}:{json.dumps(action_input, sort_keys=True)}"

        if action_key in self.action_history:
            return True  # Loop detected!

        self.action_history.append(action_key)
        return False
```

```
    def run(self, question: str, verbose: bool = True) -> str:
        """
        Run the ReAct loop to answer a question.

        Args:
            question: The user's question
            verbose: Whether to print the reasoning trace

        Returns:
            The agent's final answer
        """
        self.scratchpad = []
        self.action_history = []

        system_prompt = self._build_system_prompt()

        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": question}
        ]

        if verbose:
            print(f"\n{'='*60}")
            print(f"Question: {question}")
            print(f"{'='*60}")

        for step in range(1, self.max_steps + 1):
            if verbose:
                print(f"\n--- Step {step}/{self.max_steps} ---")

            # Call LLM
            response = self._call_llm(messages)

            if verbose:
                print(response)

            # Parse response
            parsed = self._parse_response(response)

            # Check for final answer
            if parsed["final_answer"]:
                if verbose:
                    print(f"\n{'='*60}")
                    print(f"Final Answer: {parsed['final_answer']}")
                    print(f"Steps taken: {step}")
                    print(f"{'='*60}")
                return parsed["final_answer"]

            # Validate action
            if not parsed["action"]:
                messages.append({"role": "assistant", "content": response})
                messages.append({
                    "role": "user",
                    "content": "Please respond with either an Action to use a tool, "
                               "or a Final Answer if you're done."
                })
```

```
                continue

            # Check for loops
            if self._check_loop(parsed["action"], parsed["action_input"] or {}):
                messages.append({"role": "assistant", "content": response})
                messages.append({
                    "role": "user",
                    "content": f"You already tried {parsed['action']} with these "
                               f"same inputs. Please try a DIFFERENT approach."
                })
                if verbose:
                    print("⚠ Loop detected! Forcing different approach.")
                continue

            # Execute tool
            observation = self._execute_tool(
                parsed["action"],
                parsed["action_input"] or {}
            )

            if verbose:
                print(f"Observation: {observation}")

            # Add to conversation
            self.scratchpad.append({
                "step": step,
                "thought": parsed["thought"],
                "action": parsed["action"],
                "action_input": parsed["action_input"],
                "observation": observation
            })

            messages.append({"role": "assistant", "content": response})
            messages.append({
                "role": "user",
                "content": f"Observation: {observation}"
            })

        # Max steps reached
        return ("I was unable to fully answer the question within the "
                f"maximum {self.max_steps} steps. Here's what I found so far: "
                + self._summarize_scratchpad())

    def _call_llm(self, messages: list) -> str:
        """Call the LLM. Override this for different providers."""
        import openai

        response = openai.chat.completions.create(
            model=self.model,
            messages=messages,
            temperature=0.1,
            max_tokens=500,
            stop=["Observation:"]  # Stop before observation (we provide it)
        )
        return response.choices[0].message.content.strip()
```

```
    def _summarize_scratchpad(self) -> str:
        """Summarize what the agent found so far."""
        if not self.scratchpad:
            return "No information gathered."

        summary = []
        for entry in self.scratchpad:
            summary.append(f"- Used {entry['action']}: {entry['observation'][:100]}")
        return "\n".join(summary)


# ============================================================
# SECTION 3: STANDALONE VERSION (No API key needed)
# ============================================================

class StandaloneReActAgent:
    """
    A ReAct agent that works WITHOUT an LLM API.
    Uses rule-based reasoning for practice/learning.

    This demonstrates the ReAct pattern clearly without
    requiring any API keys or external services.
    """

    def __init__(self, tools: dict, max_steps: int = 10):
        self.tools = tools
        self.max_steps = max_steps
        self.scratchpad = []
        self.action_history = []

    def run(self, question: str, plan: list = None) -> str:
        """
        Run the agent with a predefined plan.

        Args:
            question: User's question
            plan: List of (action, action_input) tuples
                  If None, agent tries to create one from the question
        """
        self.scratchpad = []
        self.action_history = []

        print(f"\n{'='*60}")
        print(f"Question: {question}")
        print(f"{'='*60}")

        if plan is None:
            plan = self._simple_planner(question)

        print(f"\nPlan: {len(plan)} steps")
        for i, (action, inputs) in enumerate(plan, 1):
            print(f"  {i}. {action}({json.dumps(inputs)})")

        # Execute plan
        results = []
        for step, (action, action_input) in enumerate(plan, 1):
```

```
            print(f"\n--- Step {step} ---")
            print(f"Thought: Executing step {step} of the plan")
            print(f"Action: {action}")
            print(f"Action Input: {json.dumps(action_input)}")

            # Check for loops
            action_key = f"{action}:{json.dumps(action_input, sort_keys=True)}"
            if action_key in self.action_history:
                print("⚠ Loop detected! Skipping duplicate action.")
                continue
            self.action_history.append(action_key)

            # Execute
            if action not in self.tools:
                observation = f"Error: Tool '{action}' not found"
            else:
                try:
                    observation = self.tools[action].function(action_input)
                except Exception as e:
                    observation = f"Error: {e}"

            print(f"Observation: {observation}")
            results.append({"action": action, "result": observation})

            self.scratchpad.append({
                "step": step,
                "action": action,
                "input": action_input,
                "observation": observation
            })

        # Compile final answer
        answer = self._compile_answer(question, results)
        print(f"\n{'='*60}")
        print(f"Final Answer: {answer}")
        print(f"{'='*60}")
        return answer

    def _simple_planner(self, question: str) -> list:
        """Create a basic plan from keywords in the question."""
        plan = []
        q = question.lower()

        # Detect calculation needs
        if any(op in q for op in ["+", "-", "*", "/", "calculate", "compute", "sum"]):
            # Extract math expression
            expr = re.findall(r'[\d\s+\-*/().]+', question)
            if expr:
                plan.append(("calculator", {"expression": expr[0].strip()}))

        # Detect weather needs
        if "weather" in q:
            cities = ["tokyo", "new york", "london", "paris", "sydney"]
            for city in cities:
                if city in q:
                    plan.append(("get_weather", {"city": city}))
```

```
        # Detect search needs
        if any(w in q for w in ["who", "what", "population", "creator", "largest"]):
            plan.append(("search", {"query": question}))

        if not plan:
            plan.append(("search", {"query": question}))

        return plan

    def _compile_answer(self, question: str, results: list) -> str:
        """Compile results into a final answer."""
        parts = []
        for r in results:
            if "Error" not in r["result"]:
                parts.append(r["result"])

        if parts:
            return " | ".join(parts)
        return "Could not find an answer to your question."


# ═══════════════════════════════════════════════════════════════
# SECTION 4: RUN THE AGENT
# ═══════════════════════════════════════════════════════════════

def demo_standalone():
    """Demo the standalone agent (no API key needed)."""
    agent = StandaloneReActAgent(tools=TOOLS)

    # Test 1: Math calculation
    agent.run(
        "What is 47 * 83 + 156?",
        plan=[
            ("calculator", {"expression": "47 * 83"}),
            ("calculator", {"expression": "3901 + 156"}),
        ]
    )

    print("\n" + "─"*60 + "\n")

    # Test 2: Weather query
    agent.run(
        "What's the weather in Tokyo?",
        plan=[
            ("get_weather", {"city": "Tokyo"}),
        ]
    )

    print("\n" + "─"*60 + "\n")

    # Test 3: Multi-tool task
    agent.run(
        "Compare the weather in Tokyo and London, and calculate "
        "the temperature difference.",
        plan=[
```

```
            ("get_weather", {"city": "Tokyo"}),
            ("get_weather", {"city": "London"}),
            ("calculator", {"expression": "22 - 12"}),
        ]
    )

    print("\n" + "−"*60 + "\n")

    # Test 4: Search + calculation
    agent.run(
        "What is the population of Tokyo?",
        plan=[
            ("search", {"query": "population tokyo"}),
        ]
    )

    print("\n" + "−"*60 + "\n")

    # Test 5: Auto-plan (no predefined plan)
    agent.run("What's the weather in Paris?")


def demo_with_llm():
    """Demo with a real LLM (requires OPENAI_API_KEY)."""
    agent = ReActAgent(tools=TOOLS, model="gpt-4", max_steps=10)

    # Test complex multi-step reasoning
    answer = agent.run(
        "What is the weather in Tokyo? If the temperature is above 20°C, "
        "calculate how many degrees above 20 it is, then tell me the result."
    )
    print(f"\nResult: {answer}")


if __name__ == "__main__":
    print("Running standalone demo (no API key needed)...")
    print("=" * 60)
    demo_standalone()

    # Uncomment to run with real LLM:
    # demo_with_llm()
```

Exercise: Extend the Agent

Try these modifications to deepen your understanding:

```
1. ADD A NEW TOOL
   Create a "unit_converter" tool that converts between:
   - Celsius ↔ Fahrenheit
   - Kilometers ↔ Miles
   - Kilograms ↔ Pounds
   Add it to the TOOLS dict with proper schema.
```

```
2. ADD EPISODIC MEMORY
   Create a simple log file that records:
   - Each task the agent received
   - What tools it used
   - Whether it succeeded
   Load this log at startup so the agent can learn from past runs.

3. ADD A TIMEOUT
   If a tool takes more than 5 seconds, return an error
   and let the agent try a different approach.

4. ADD CONTEXT WINDOW MANAGEMENT
   When the scratchpad gets too long (e.g., > 2000 tokens),
   summarize earlier steps into a compact format.
```

✓ Day 3 Deliverable Checklist

- ☐ Built a complete ReAct agent with at least 3 tools
- ☐ Agent can plan multi-step reasoning
- ☐ Agent detects and avoids action loops
- ☐ Agent validates tool names and arguments
- ☐ Agent handles tool errors gracefully
- ☐ Tested with at least 3 different types of questions
- ☐ Can explain: What is the ReAct pattern and why is it effective?
- ☐ Can explain: What are the three types of agent memory?
- ☐ Can explain: What are common agent failure modes and how to prevent them?

Next → Day 4: Planner/Executor Architecture